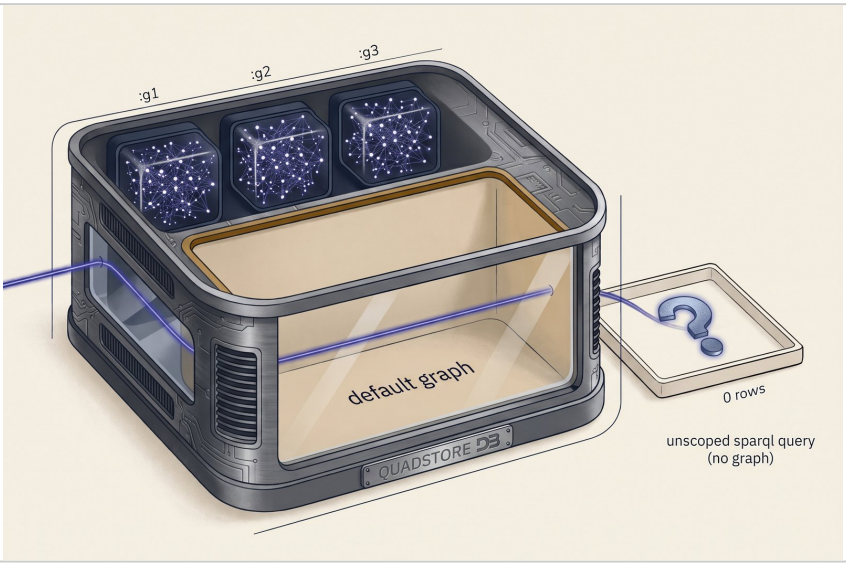
Graph Composition Within the RDF Technologies Ecosystem



A Holonic Approach

Zachary Welz, PhD (Peerless Tech.)

2026-05-06

How holonic named-graph architecture solves the federation, governance, and provenance problems that RDF has left unaddressed for over a decade.

Promise vs. Reality

RDF’s data model is deceptively elegant. Every resource has a URI. Two graphs mentioning the same URI? Merge them and the nodes unify automatically. This is the foundational integration promise of the Semantic Web. In tutorials, it works beautifully. In production it’s a bit more complicated…

Production RDF deployments consist of quad stores managing hundreds of named graphs, often distributed across organizational boundaries, with vocabulary heterogeneity and governance requirements that the simple union model never addressed. There are three primary blockers.

Problem 1: The Default Graph Divergence

The SPARQL spec defines a “default graph” but is curiously silent on what it contains. Different quad stores end up interpreting this differently: Fuseki (union mode) treats it as the join of all named graphs, GraphDB treats it as empty, and Stardog makes it configurable. The same bare triple pattern query returns different results depending on which store you’re running.

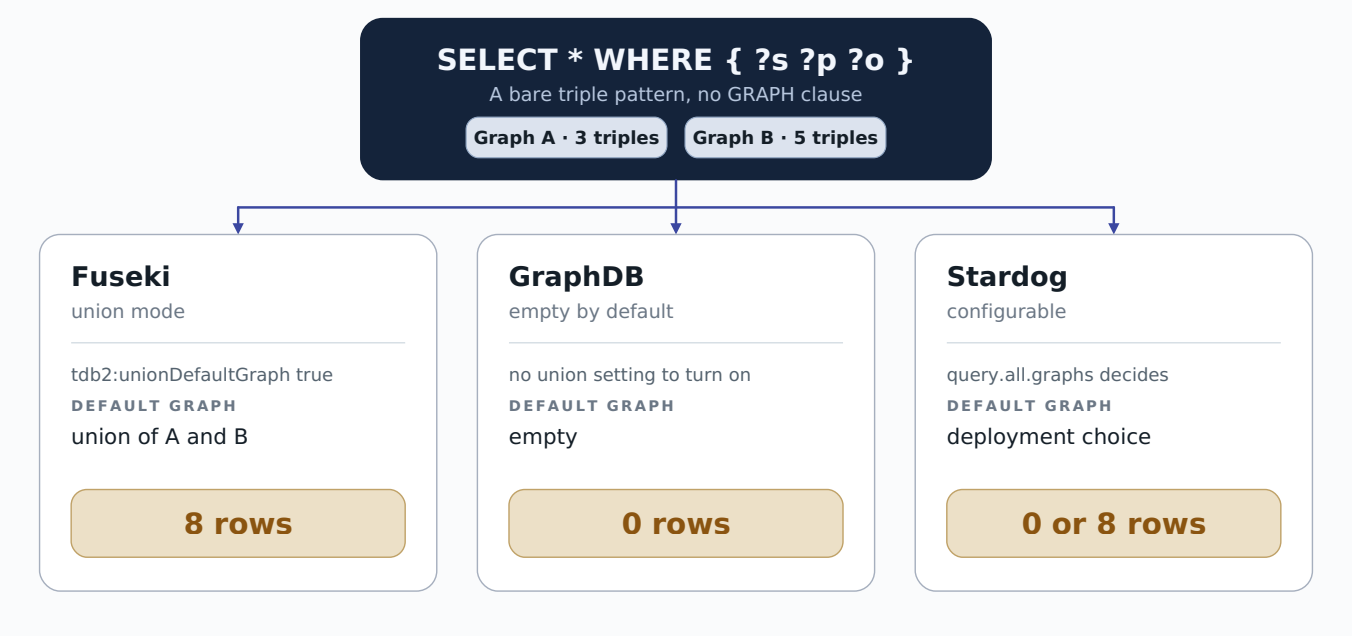


Figure 1. Same data, same query, three different answers. The SPARQL specification is non-prescriptive on default graph semantics.

This means that the “just union the graphs” strategy is not portably available as a default behavior. You cannot write a SPARQL query and expect it to return the same results across implementations without explicit GRAPH clauses.

Problem 2: Federated Query Variable Scoping

When data lives in separate SPARQL endpoints, the standard integration mechanism is SERVICE federation. The expectation is that bound variables in the outer query propagate into the SERVICE block. In practice, the SPARQL 1.1 spec says implementations “SHOULD” push bindings, but does not mandate it.

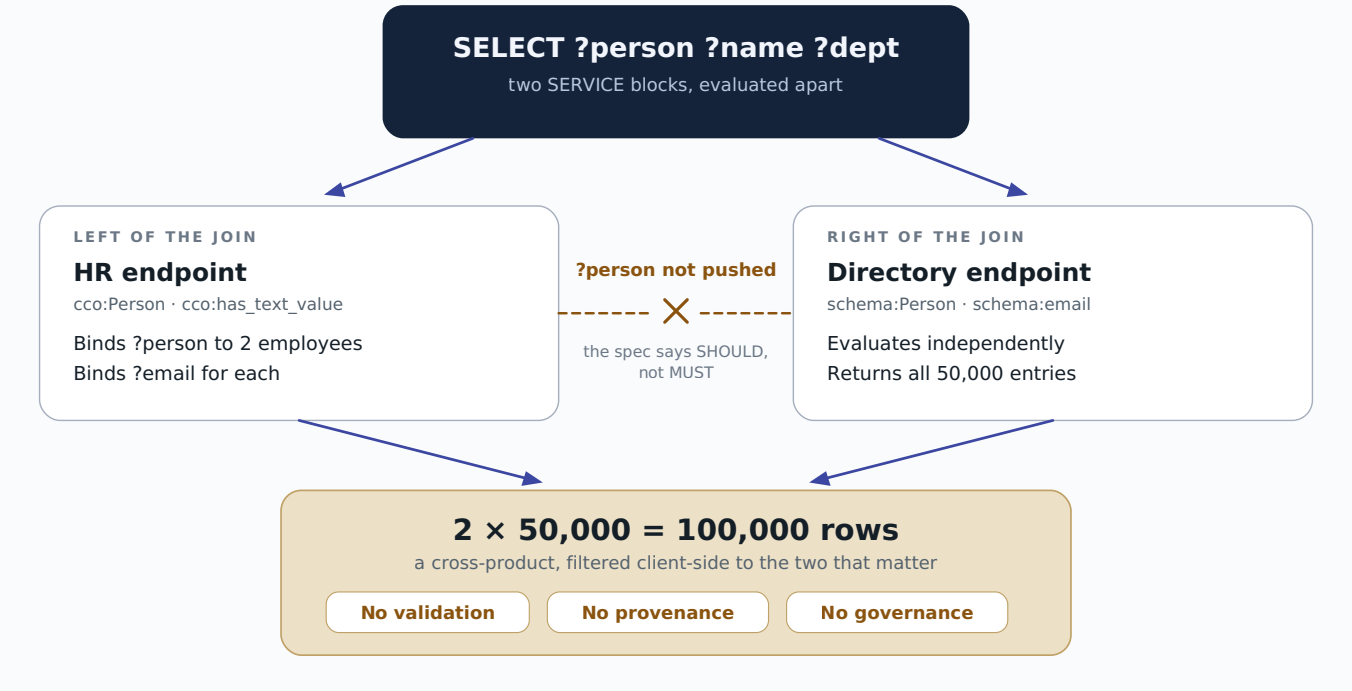


Figure 2. The SERVICE variable scoping failure. Bindings from the left endpoint may not be pushed to the right, producing a cross-product instead of a join.

The result: if the HR system binds 2 employees and the Directory has 50,000 entries, you get a 100,000-row cross-product that must be filtered client-side. Beyond the performance cost, there is no validation of the data shape, no provenance of who queried what and when, and no governance layer controlling what crosses system boundaries.

Problem 3: Band-Aids Don’t Scale

Materialized views and ETL pipelines work but abandon the live integration promise. The snapshot drifts from the source. **VALUES injection** works for small result sets but chokes on 10,000 URIs. **owl:sameAs inference** shifts the problem from query time to reasoning time and introduces the transitive-closure “sameAs problem.” **Named graph conventions** are fragile and don’t extend across endpoints.

None of these provides a principled, scalable, governance-aware mechanism for composing information from multiple named graphs while preserving provenance and enforcing data contracts.

The Holonic Approach

The holonic library implements Cagel’s four-graph holon model. Drawing on Arthur Koestler’s concept of the holon, an entity that is simultaneously a self-contained whole and a part of a larger system, it organizes knowledge into governed units with four named-graph layers.

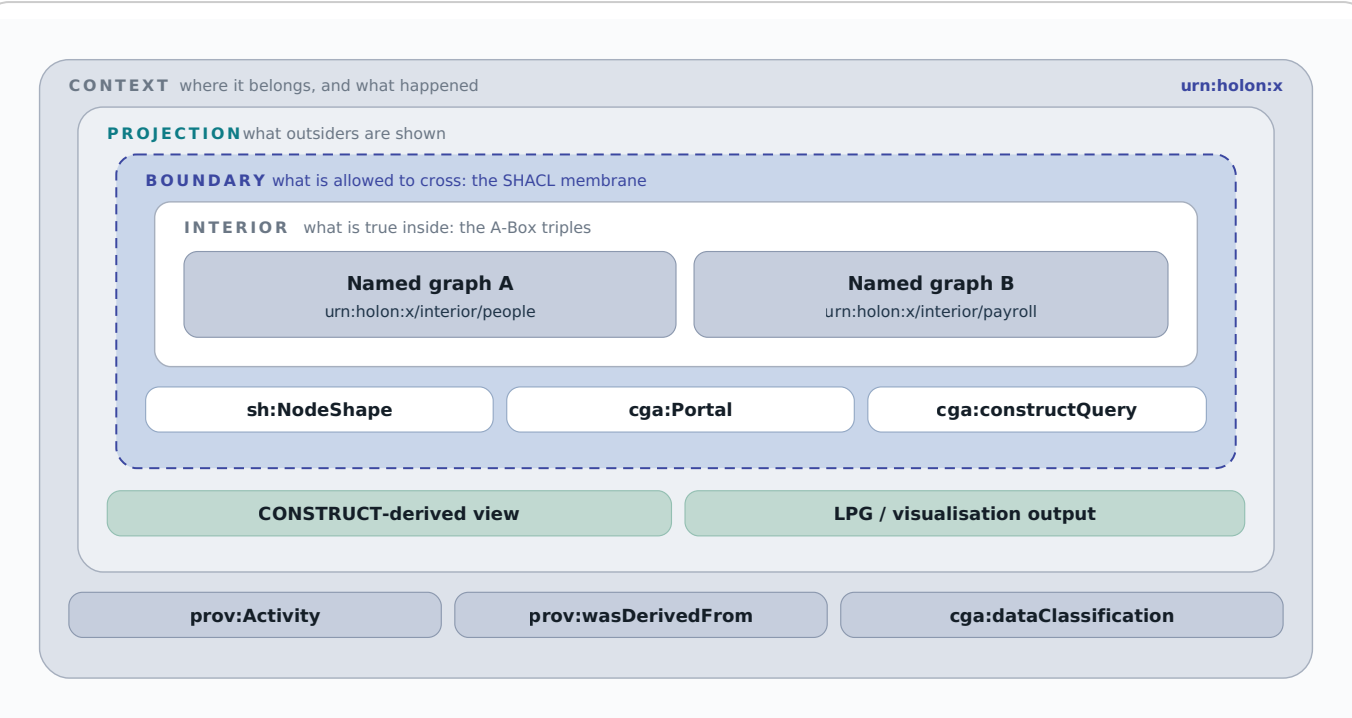


Figure 3. The four-graph holon: Interior (what is true), Boundary (what is allowed), Projection (what outsiders see), Context (where it belongs and what happened).

Interior: The A-Box triples, the facts. A holon can have multiple interior named graphs (e.g., contacts and payroll), treated as a logical union within the holon's scope. **Boundary:** SHACL shapes defining what data can cross the membrane, plus portal definitions that govern inter-holon data flow. **Projection:** A CONSTRUCT-derived or filtered view of what downstream consumers are allowed to see. **Context:** PROV-O provenance activities, temporal annotations, membership, and governance metadata.

Portals: Governed Graph-to-Graph Data Flow

Holons are connected by portals, first-class RDF entities stored in boundary graphs. A TransformPortal carries a SPARQL CONSTRUCT query that translates data from the source vocabulary to the target vocabulary. Traversal executes the CONSTRUCT, validates the result against the target's SHACL boundary shapes, and records a prov:Activity in the target's context graph.

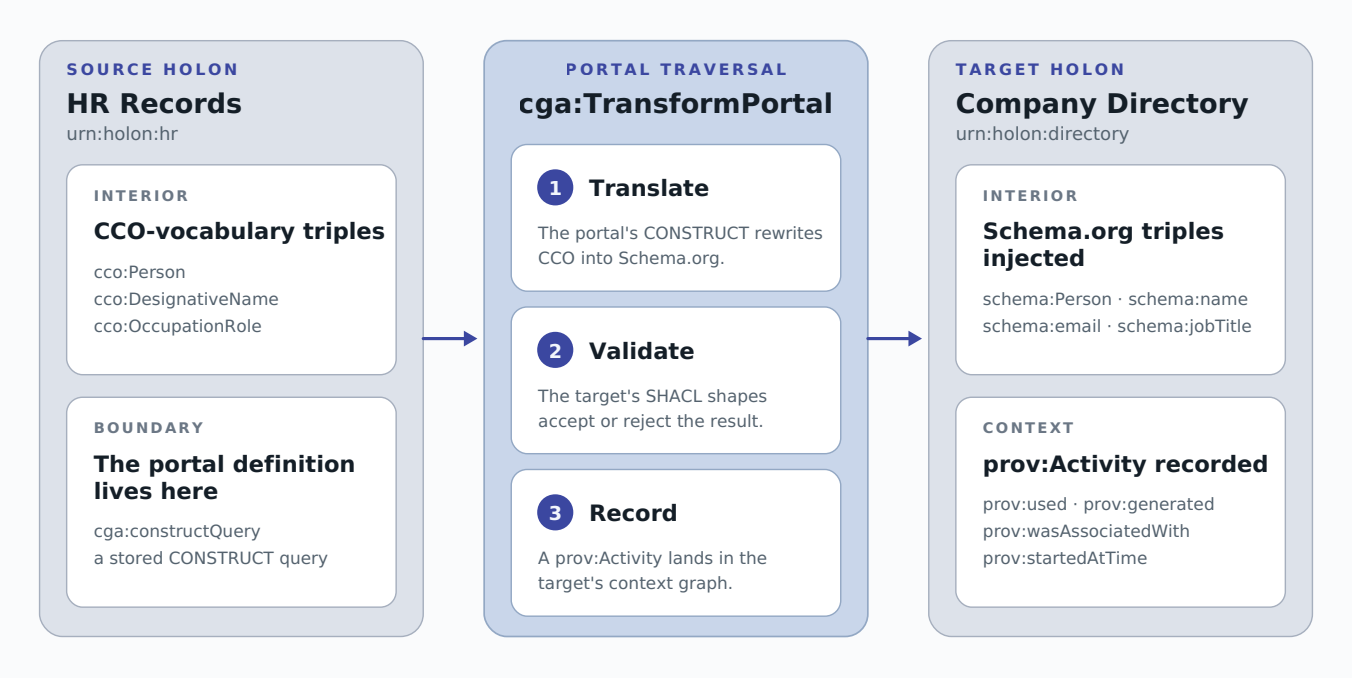


Figure 4. Portal traversal: CONSTRUCT translates, SHACL validates, PROV-O records.

This is the key reframing: instead of one monolithic cross-endpoint query, graph composition becomes a sequence of bounded, governed, provenance-tracked data movements. Each portal traversal operates on the source holon's interior graphs (a small, local union), not the entirety of the enterprise knowledge graph.

Worked Example: Three-Holon Pipeline

Consider an organization with HR Records (CCO vocabulary), a Company Directory (Schema.org), and an Analytics Warehouse. The goal: make employee data available in Analytics with full provenance and validation at each boundary.

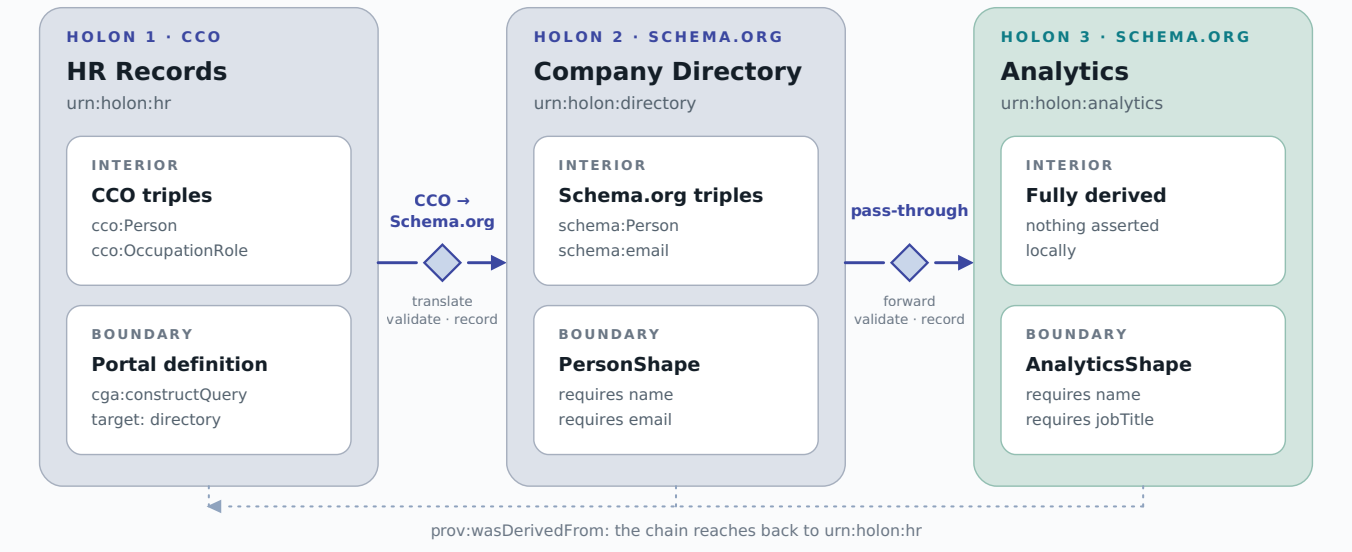


Figure 5. HR → Directory → Analytics. Two portals, two vocabulary translations, two boundary validations, full provenance chain.

The first portal's CONSTRUCT query maps CCO person entities to Schema.org. The second portal forwards the already-translated data to Analytics. At each step, SHACL shapes validate the injected data (the Directory requires name and email; Analytics requires name and jobTitle), and a prov:Activity records exactly what was derived from where.

Contrast this with the federated SPARQL approach: the query must know both CCO and Schema.org vocabularies, variables may not bind across SERVICE blocks, there is no validation, no provenance, no governance, and the cross-product can explode at scale.

The code is concise

Declaring holons, populating interiors, registering portals with CONSTRUCT queries, and running governed traversal requires only a few dozen lines of Python. Each portal's CONSTRUCT query is stored as RDF in the boundary graph: inspectable, versioned, and discoverable via SPARQL.

```
projected, membrane = ds.traverse(
    "urn:holon:hr", "urn:holon:directory",
    validate=True, agent_iri="urn:agent:pipeline")
# membrane.health -> Intact | Weakened | Compromised
```

Federation and Virtualization Still Fit

Holonic doesn't displace existing infrastructure. It wraps it. A Virtual Knowledge Graph (like Ontop backed by PostgreSQL) can serve as a holon's interior, either via a custom backend that routes SPARQL to the VKG endpoint, or via a materialized snapshot refreshed on schedule.

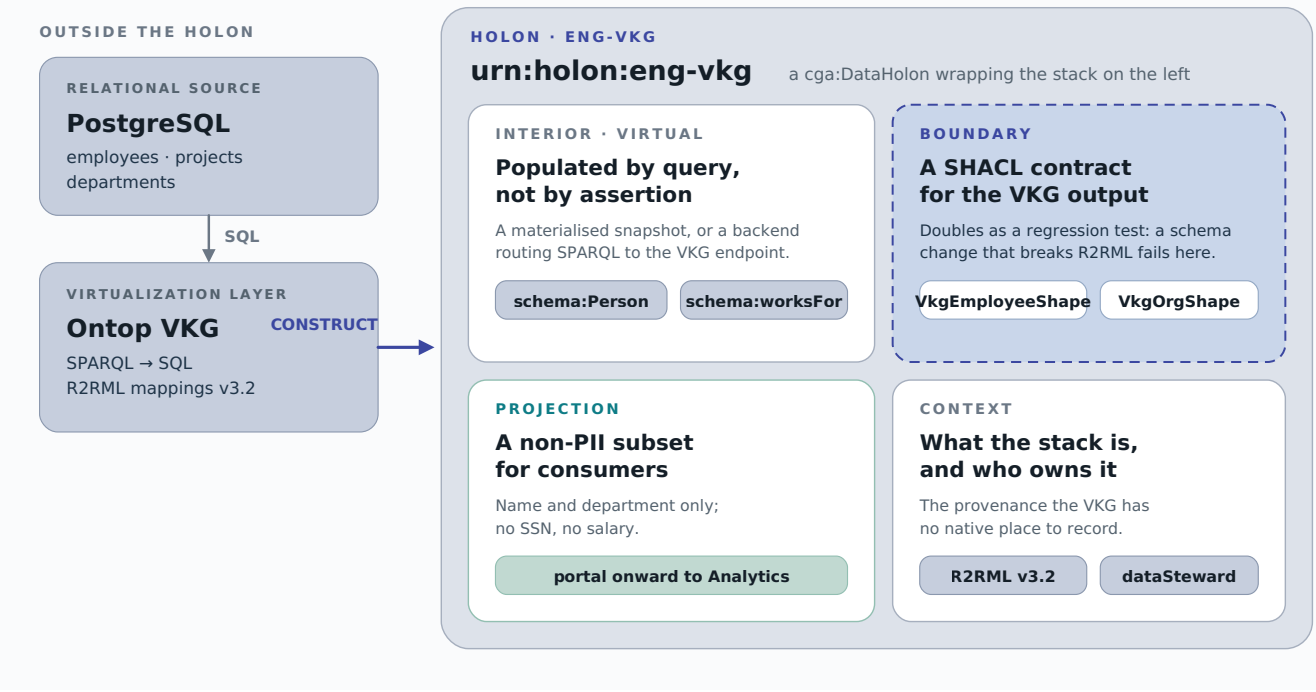


Figure 6. PostgreSQL + Ontop wrapped as a governed holon. The interior is virtual; the governance layers are real.

The boundary SHACL shapes become a regression test for the R2RML mappings: if a PostgreSQL schema change breaks the mapping, membrane validation catches it. The context graph records the Ontop endpoint URL, the R2RML version, the PG schema version, the data steward, and the classification level. The projection filters to a non-PII subset for downstream consumers.

The same pattern extends to any SERVICE endpoint: Wikidata, DBpedia, a GeoSPARQL service. The holon wrapper adds the governance frame (boundary contracts, provenance, classification) that these technologies lack natively.

Challenges and Trade-Offs

Materialization cost. Portal traversal writes triples into the target interior. This is a snapshot that drifts from the source between refresh cycles. But this is the same trade-off every ETL pipeline makes. Holonic simply brings it under governance with provenance and validation.

Portal query complexity. Cross-vocabulary CONSTRUCT queries can be intricate. The library supports projection pipelines (composable sequences of CONSTRUCT and Python transforms) to manage this.

Registration cost. Converting an existing quad store into a holarchy requires explicit holon declaration, portal registration, and boundary definition. This is by design: governance structure (who stewards what, at what classification level) is not inferrable from data alone.

The Living Graph

The graph composition problem in RDF is not a theoretical limitation. It is a practical consequence of how named graphs, quad stores, and federation interact in production. Default graph divergence, unreliable variable scoping, and missing governance make naive union inadequate for enterprise knowledge graphs.

Holonic reframes the problem: decompose the knowledge space into governed holons connected by portal-mediated data flows. Each traversal is bounded, validated, provenance-tracked. Existing federation and virtualization infrastructure can be wrapped as holons with boundary contracts, context provenance, and projection controls.

The living graph is not a single, monolithic union. It is a holarchy of governed parts, each whole in itself.

References

1. Cagel, K. “The Living Graph: Holons and the Four-Graph Model.” The Ontologist, March 2026.

2. Koestler, A. The Ghost in the Machine. Hutchinson, 1967.

3. W3C. SPARQL 1.1 Query Language / Federated Query. W3C Recommendation, 2013.

4. W3C. Shapes Constraint Language (SHACL). W3C Recommendation, 2017.

holonic is open source:

pip install holonic

Zachary Welz, PhD